



Contract 0x5FF137D4b0FDCD49DcA30c7CF57E578a026d2789



Buy ▾

Play ▾

Gaming ▾

Sponsored:  **Mentorable:** trade creators, monetize expertise, earn forever. [Turn your skills into a liquid market.](#)

This smart contract is used as an entry point for ERC-4337 Account Abstraction transactions.

Entry Point 0.6.0 

[Source Code](#)



</> API



Overview

ETH BALANCE

311.150915215750768275 ETH

ETH VALUE

\$918,850.62 (@ \$2,953.07/ETH)

TOKEN HOLDINGS

>\$315.49 (>201 Tokens) ⓘ ▾



More Info

PRIVATE NAME TAGS

madib22



CREATOR

0x433704c4...5e31eB0c1 

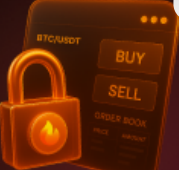
2 yrs 181
days ago
↗

Multichain Info

\$2,081,844.03 (Multichain Portfolio)

28 addresses found via [Blockscan](#) ▾

 **HIBACHI**
For perps traders
who'd rather be
profitable than visible
[Trade Privately](#)



Ad

Ad

[Transactions](#)[Internal Transactions](#)[Token Transfers \(ERC-20\)](#)[NFT Transfers](#)[Other Transactions](#)[🔍 Advanced Filter](#)[Code](#)[Read Contract](#)[Write Contract](#)

✔ **Contract Source Code Verified** (Exact Match)



Contract Name: **EntryPoint**

Optimization

Yes with **1000000** runs

Enabled:

Compiler

v0.8.17+commit.8df45

Version

f5f

Other Settings:

default evmVersion

📄 **Contract Source Code** (Solidity [Standard Json-Input](#) format)

IDE ▾

More Options ▾

File 1 of 14 : EntryPoint.sol

Outline ▾



```
1  /**
2   ** Account-Abstraction (EIP-4337) singleton EntryPoint implementation.
3   ** Only one instance required on each chain.
4   **/
5   // SPDX-License-Identifier: GPL-3.0
6   pragma solidity ^0.8.12;
7
8   /* solhint-disable avoid-low-level-calls */
9   /* solhint-disable no-inline-assembly */
10
11  import "../interfaces/IAccount.sol";
12  import "../interfaces/IPaymaster.sol";
13  import "../interfaces/IEntryPoint.sol";
14
15  import "../utils/Exec.sol";
16  import "../StakeManager.sol";
17  import "../SenderCreator.sol";
18  import "../Helpers.sol";
19  import "../NonceManager.sol";
20  import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
21
```

```

22 contract EntryPoint is IEntryPoint, StakeManager, NonceManager, Reentrant
23
24     using UserOperationLib for UserOperation;

```

File 2 of 14 : ReentrancyGuard.sol

Outline ▾



```

1  // SPDX-License-Identifier: MIT
2  // OpenZeppelin Contracts (last updated v4.8.0) (security/ReentrancyGuard
3
4  pragma solidity ^0.8.0;
5
6  /**
7   * @dev Contract module that helps prevent reentrant calls to a function.
8   *
9   * Inheriting from `ReentrancyGuard` will make the {nonReentrant} modifier
10  * available, which can be applied to functions to make sure there are no
11  * (reentrant) calls to them.
12  *
13  * Note that because there is a single `nonReentrant` guard, functions marked
14  * `nonReentrant` may not call one another. This can be worked around by
15  * those functions `private`, and then adding `external` `nonReentrant` entry
16  * points to them.
17  *
18  * TIP: If you would like to learn more about reentrancy and alternative ways
19  * to protect against it, check out our blog post
20  * https://blog.openzeppelin.com/reentrancy-after-istanbul/[Reentrancy After
21  */
22  abstract contract ReentrancyGuard {
23      // Booleans are more expensive than uint256 or any type that takes up
24      // word because each write operation emits an extra SLOAD to first read
25      // slot's contents, replace the bits taken up by the boolean, and then

```

File 3 of 14 : Helpers.sol

Outline ▾



```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity ^0.8.12;
3
4  /* solhint-disable no-inline-assembly */
5
6  /**
7   * returned data from validateUserOp.
8   * validateUserOp returns a uint256, with is created by `_packedValidationData`
9   * parsed by `_parseValidationData`
10  * @param aggregator - address(0) - the account validated the signature by
11  * address(1) - the account failed to validate the signature
12  * otherwise - this is an address of a signature aggregator
13  * used to validate the signature.
14  * @param validAfter - this UserOp is valid only after this timestamp.
15  * @param validUntil - this UserOp is valid only up to this timestamp.
16  */
17  struct ValidationData {

```

```

16         address aggregator;
17         uint48 validAfter;
18         uint48 validUntil;
19     }
20
21     //extract sigFailed, validAfter, validUntil.
22     // also convert zero validUntil to type(uint48).max
23     function parseValidationData(uint validationData) pure returns (Vali

```

File 4 of 14 : NonceManager.sol

Outline ▾



```

1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.12;
3
4 import "../interfaces/IEntryPoint.sol";
5
6 /**
7  * nonce management functionality
8  */
9 contract NonceManager is INonceManager {
10
11     /**
12      * The next valid sequence number for a given nonce key.
13      */
14     mapping(address => mapping(uint192 => uint256)) public nonceSequenceN
15
16     function getNonce(address sender, uint192 key)
17     public view override returns (uint256 nonce) {
18         return nonceSequenceNumber[sender][key] | (uint256(key) << 64);
19     }
20
21     // allow an account to manually increment its own nonce.
22     // (mainly so that during construction nonce can be made non-zero,
23     // to "absorb" the gas cost of first nonce increment to 1st transacti
24     (construction),
25     // not to 2nd transaction)

```

File 5 of 14 : SenderCreator.sol

Outline ▾



```

1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.12;
3
4 /**
5  * helper contract for EntryPoint, to call userOp.initCode from a "neutra
6  * which is explicitly not the EntryPoint itself.
7  */
8 contract SenderCreator {
9
10     /**
11      * call the "initCode" factory to create and return the sender account
12      * @param initCode the initCode value from a UserOp. contains 20 byte.

```

```

13 address, followed by calldata
   * @return sender the returned address of the created account, or zero
   failure.
14 */
15 function createSender(bytes calldata initCode) external returns (address) {
16     address factory = address(bytes20(initCode[0 : 20]));
17     bytes memory initCallData = initCode[20 :];
18     bool success;
19     /* solhint-disable no-inline-assembly */
20     assembly {
21         success := call(gas(), factory, 0, add(initCallData, 0x20), mload(
(initCallData), 0, 32)
22         sender := mload(0)

```

File 6 of 14 : StakeManager.sol

Outline ▾



```

1  // SPDX-License-Identifier: GPL-3.0-only
2  pragma solidity ^0.8.12;
3
4  import "../interfaces/IStakeManager.sol";
5
6  /* solhint-disable avoid-low-level-calls */
7  /* solhint-disable not-rely-on-time */
8  /**
9   * manage deposits and stakes.
10   * deposit is just a balance used to pay for UserOperations (either by a
   an account)
11   * stake is value locked for at least "unstakeDelay" by a paymaster.
12   */
13  abstract contract StakeManager is IStakeManager {
14
15      /// maps paymaster to their deposits and stakes
16      mapping(address => DepositInfo) public deposits;
17
18      /// @inheritdoc IStakeManager
19      function getDepositInfo(address account) public view returns (Deposit
info) {
20          return deposits[account];
21      }
22
23      // internal method to return just the stake info

```

File 7 of 14 : IAccount.sol

Outline ▾



```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity ^0.8.12;
3
4  import "../UserOperation.sol";
5
6  interface IAccount {
7

```

```

8  ▾ /**
9      * Validate user's signature and nonce
10     * the entryPoint will make the call to the recipient only if this va
11     returns successfully.
12     * signature failure should be reported by returning SIG_VALIDATION_F
13     * This allows making a "simulation call" without a valid signature
14     * Other failures (e.g. nonce mismatch, or invalid signature format)
15     revert to signal failure.
16     *
17     * @dev Must validate caller is the entryPoint.
18     *      Must validate the signature and nonce
19     * @param userOp the operation that is about to be executed.
20     * @param userOpHash hash of the user's request data. can be used as
21     signature.
22     * @param missingAccountFunds missing funds on the account's deposit
23     entrypoint.
24     *      This is the minimum amount to transfer to the sender(entryPoi

```

File 8 of 14 : IAggregator.sol

Outline ▾



```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity ^0.8.12;
3
4  import "./UserOperation.sol";
5
6  ▾ /**
7      * Aggregated Signatures validator.
8      */
9  ▾ interface IAggregator {
10
11     ▾ /**
12         * validate aggregated signature.
13         * revert if the aggregated signature does not match the given list o
14         */
15         function validateSignatures(UserOperation[] calldata userOps, bytes c
16         signature) external view;
17
18     ▾ /**
19         * validate signature of a single userOp
20         * This method is should be called by bundler after EntryPoint.simula
21         returns (reverts) with ValidationResultWithAggregation
22         * First it validates the signature over the userOp. Then it returns
23         when creating the handleOps.
24         * @param userOp the userOperation received from the user.
25         * @return sigForUserOp the value to put into the signature field of

```

File 9 of 14 : IEntryPoint.sol

Outline ▾



```

1  ▾ /**
2      ** Account-Abstraction (EIP-4337) singleton EntryPoint implementation.

```

```

3  ** Only one instance required on each chain.
4  **/
5  // SPDX-License-Identifier: GPL-3.0
6  pragma solidity ^0.8.12;
7
8  /* solhint-disable avoid-low-level-calls */
9  /* solhint-disable no-inline-assembly */
10 /* solhint-disable reason-string */
11
12 import "./UserOperation.sol";
13 import "./IStakeManager.sol";
14 import "./IAggregator.sol";
15 import "./INonceManager.sol";
16
17 interface IEntryPoint is IStakeManager, INonceManager {
18
19     /**
20      * An event emitted after each successful request
21      * @param userOpHash - unique identifier for the request (hash its e
except signature).
22      * @param sender - the account that generates this request.
23      * @param paymaster - if non-null, the paymaster that pays for this

```

File 10 of 14 : INonceManager.sol

Outline ▾



```

1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity ^0.8.12;
3
4  interface INonceManager {
5
6      /**
7       * Return the next nonce for this sender.
8       * Within a given key, the nonce values are sequenced (starting with
incremented by one on each userop)
9       * But UserOp with different keys can come with arbitrary order.
10      *
11      * @param sender the account address
12      * @param key the high 192 bit of the nonce
13      * @return nonce a full nonce to pass for next UserOp with this sender
14      */
15     function getNonce(address sender, uint192 key)
16         external view returns (uint256 nonce);
17
18     /**
19      * Manually increment the nonce of the sender.
20      * This method is exposed just for completeness..
21      * Account does NOT need to call it, neither during validation, nor e
22      * as the EntryPoint will update the nonce regardless.
23      * Possible use-case is call it with various keys to "initialize" the
one, so that future

```

```

1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.12;
3
4 import "./UserOperation.sol";
5
6 /**
7  * the interface exposed by a paymaster contract, who agrees to pay the gas
8  * operations.
9  * a paymaster must hold a stake to cover the required entrypoint stake and
10  * for the transaction.
11  */
12 interface IPaymaster {
13     enum PostOpMode {
14         opSucceeded, // user op succeeded
15         opReverted, // user op reverted. still has to pay for gas.
16         postOpReverted //user op succeeded, but caused postOp to revert.
17         call, after user's op was deliberately reverted.
18     }
19
20     /**
21     * payment validation: check if paymaster agrees to pay.
22     * Must verify sender is the entryPoint.
23     * Revert to reject this request.
24     * Note that bundlers will reject this method if it changes the state

```

```

1 // SPDX-License-Identifier: GPL-3.0-only
2 pragma solidity ^0.8.12;
3
4 /**
5  * manage deposits and stakes.
6  * deposit is just a balance used to pay for UserOperations (either by a
7  * an account)
8  * stake is value locked for at least "unstakeDelay" by the staked entity
9  */
10 interface IStakeManager {
11     event Deposited(
12         address indexed account,
13         uint256 totalDeposit
14     );
15
16     event Withdrawn(
17         address indexed account,
18         address withdrawAddress,
19         uint256 amount
20     );
21

```



```
22     /// Emitted when stake or unstake delay are modified
23     event StakeLocked(
```

File 13 of 14 : UserOperation.sol

Outline ▾



```
1  // SPDX-License-Identifier: GPL-3.0
2  pragma solidity ^0.8.12;
3
4  /* solhint-disable no-inline-assembly */
5
6  import {calldataKeccak} from "../core/Helpers.sol";
7
8  /**
9   * User Operation struct
10   * @param sender the sender account of this request.
11   * @param nonce unique value the sender uses to verify it is not a re
12   * @param initCode if set, the account contract will be created by th
13   * @param callData the method call to execute on this account.
14   * @param callGasLimit the gas limit passed to the callData method ca
15   * @param verificationGasLimit gas used for validateUserOp and
16   validatePaymasterUserOp.
17   * @param preVerificationGas gas not calculated by the handleOps metho
18   to the gas paid. Covers batch overhead.
19   * @param maxFeePerGas same as EIP-1559 gas parameter.
20   * @param maxPriorityFeePerGas same as EIP-1559 gas parameter.
21   * @param paymasterAndData if set, this field holds the paymaster add
22   paymaster-specific data. the paymaster will pay for the transaction inste
23   sender.
24   * @param signature sender-verified signature over the entire request
25   EntryPoint address and the chain ID.
```

File 14 of 14 : Exec.sol

Outline ▾



```
1  // SPDX-License-Identifier: LGPL-3.0-only
2  pragma solidity >=0.7.5 <0.9.0;
3
4  // solhint-disable no-inline-assembly
5
6  /**
7   * Utility functions helpful when making different kinds of contract call
8   */
9  library Exec {
10
11     function call(
12         address to,
13         uint256 value,
14         bytes memory data,
15         uint256 txGas
16     ) internal returns (bool success) {
17         assembly {
```

```

18         success := call(txGas, to, value, add(data, 0x20), mload(data
19     }
20 }
21
22 function staticcall(
23     address to,
24     bytes memory data,

```

Settings



```

1 {
2   "optimizer": {
3     "enabled": true,
4     "runs": 1000000
5   },
6   "viaIR": true,
7   "outputSelection": {
8     "*": {
9       "*": [
10        "evm.bytecode",
11        "evm.deployedBytecode",
12        "devdoc",
13        "userdoc",
14        "metadata",
15        "abi"
16      ]
17    }
18  },
19  "libraries": {}
20 }

```

Contract Security Audit

- ⊗ No Contract Security Audit Submitted - [Submit Audit Here](#)

Contract ABI </> API



```

[{"inputs":
[{"internalType":"uint256","name":"preOpGas","type":"uint256"},
{"internalType":"uint256","name":"paid","type":"uint256"},
{"internalType":"uint48","name":"validAfter","type":"uint48"},
{"internalType":"uint48","name":"validUntil","type":"uint48"},
{"internalType":"bool","name":"targetSuccess","type":"bool"},
{"internalType":"bytes","name":"targetResult","type":"bytes"}], "name": "
ExecutionResult", "type": "error"}, {"inputs":

```

```
[{"internalType":"uint256","name":"opIndex","type":"uint256"},  
{"internalType":"uint256","name":"opIndex","type":"uint256"},
```

</> Contract Creation Code

[Decompile Bytecode](#)[Switch to Opcodes View](#)

```
60a080604052346200008957600160025561022c8181016001600160401b03811183821  
0176200007357829162005d18833903906000f080156200006757608052604051615c89  
90816200008f823960805181818161113df01528181613e9501526141b60152f35b60405  
13d6000823e3d90fd5b634e487b7160e01b600052604160045260246000fd5b600080fd  
fe60806040526004361015610023575b361561001957600080fd5b610021615531565b0  
05b60003560e01c80630396cb60146101b35780630bd28e3b146101aa5780631b2e01b8  
146101a15780631d732756146101985780631fad948c1461018f578063205c287814610  
18657806335567e1a1461017d5780634b1d7cf5146101745780635287ce121461016b57  
806370a08231146101625780638f41ec5a14610159578063957122ab146101505780639  
b240f6014610147578063a61035311461013e578063b760fa914610135578063bb0fa6
```

</> Deployed Bytecode

```
0x60806040526004361015610023575b361561001957600080fd5b610021615531565b0  
05b60003560e01c80630396cb60146101b35780630bd28e3b146101aa5780631b2e01b8  
146101a15780631d732756146101985780631fad948c1461018f578063205c287814610
```

 A contract address hosts a smart contract, which is a set of code stored on the blockchain that runs when predetermined conditions are met. Learn more about addresses in our [Knowledge Base](#).